ABSTRACT

PhishGuard – Phishing Website Prediction Tool with Dashboard is a web-based application developed to help users identify potentially harmful and fraudulent websites. Phishing is a common cybersecurity threat in which attackers create fake websites that imitate legitimate ones to deceive users and steal sensitive information like passwords and banking details.

In addition to machine learning-based prediction, the system performs technical security checks including DNS resolution, IP address identification, HTTPS certificate verification, domain registration age analysis, and suspicious URL pattern detection.

The system is developed using Python, Flask, HTML, CSS, JavaScript, Scikit-learn, and SQLite. The application provides a detailed security report containing the classification result, confidence value, and detected security indicators. The History section allows users to review previously scanned websites, while the Reviews section enables user feedback.

Furthermore, the platform incorporates automated threat intelligence gathering to ensure real-time analysis during web evaluation. It provides intuitive visual feedback through the interactive dashboard, enabling users to assess online risks effortlessly.

The main objective of PhishGuard is to provide users with a simple, centralized, and accessible environment for analyzing suspicious websites and understanding potential phishing risks through automated security analysis.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	v
I	INTRODUCTION	1
II	SYSTEM ANALYSE	
	2.1 Existing System	6
	2.2 Proposed System	7
III	SYSTEM SPECIFICATION	
	3.1 Hardware Specification	8
	3.2 Software Specification	8
	3.3 Software Description	9
IV	SYSTEM DESIGN	
	4.1 Database Design	10
	4.2 Dataflow Diagram	11
V	SYSTEM IMPLEMENTATION	
	5.1 Module Description	12
	5.2 Form Design	13
VI	SYSTEM TESTING	18
VII	CONCLUSION AND FUTURE SCOPE	
	6.1 Conclusion	19
	6.2 Future Scope	20
	BIBILIOGRAPHY	21
	APPENDIX	22

CHAPTER – I
INTRODUCTION

The rapid growth of Internet technology has made websites an important part of daily life. People use websites for communication, education, banking, shopping, entertainment, and business. At the same time, cyber threats such as phishing attacks have also increased.

Phishing is a cyber-attack in which attackers create fake websites or links that look like genuine websites. These websites may be used to collect sensitive information such as usernames, passwords, banking details, and other personal information. Identifying such websites manually can be difficult for normal users.

PhishGuard is a web-based phishing website detection and security analysis system developed to help users identify potentially unsafe websites. The system allows users to enter a website URL and perform an automated security analysis. The submitted URL is analyzed using a machine learning model. TF-IDF character n-gram feature extraction is used to convert the URL into numerical features, and Logistic Regression is used to predict whether the website is safe or phishing.

In addition to machine learning prediction, the system performs security checks such as DNS resolution, IP address identification, HTTPS verification, domain information analysis, domain age analysis, and brand impersonation detection. The main aim of PhishGuard is to provide a simple and user-friendly platform that helps users analyze website URLs and understand their security status before accessing them.

The system also provides a user-friendly dashboard to display the analysis results in an understandable format. It presents the prediction result along with security findings, helping users identify suspicious website characteristics more easily. The system maintains scan history for future references and supports website screenshot analysis to assist in identifying potentially unsafe websites. By combining machine learning techniques with security verification methods, PhishGuard aims to improve user awareness of phishing threats and support safer browsing practices.

PROBLEM STATEMENT

Phishing websites are one of the major cybersecurity threats that deceive users by imitating legitimate websites to steal sensitive information such as usernames, passwords, banking details, and personal data. These websites often use misleading URLs, fake domain names, and similar website designs to appear genuine, making it difficult for users to identify fraudulent websites. As internet usage and online transactions continue to increase, the risk of phishing attacks has also become a significant concern.

Traditional phishing detection methods may require manual verification or depend on existing security tools and databases. These methods may not always identify newly created or unknown phishing websites. Furthermore, users without sufficient technical knowledge may find it difficult to analyse suspicious URLs and understand the security risks associated with a website. Collecting website information from different sources can also be time-consuming and inconvenient.

Therefore, there is a need for an automated and user-friendly system that can analyse websites and provide clear security results. The **PhishGuard – Phishing Website Prediction Tools with Dashboard** project is developed to address these challenges by using machine learning techniques to predict whether a submitted website is potentially legitimate or phishing. The system also aims to provide relevant website details, such as domain information, IP address, website name, domain age, discoverable pages, and website purpose, wherever the information is available.

Additionally, the system aims to support screenshot-based website analysis by examining uploaded images and attempting to identify the corresponding website for further investigation. The analysis results are presented through a centralized dashboard, enabling users to understand the website's security status more easily. Thus, the proposed system aims to simplify phishing detection, reduce the effort involved in manual verification, and help users make informed decisions before accessing potentially suspicious websites.

OBJECTIVES

The main objective of the PhishGuard system is to provide a simple and intelligent web-based platform that helps users analyze website URLs and identify potentially phishing websites.

1. The specific objectives of the system are:
2. To provide a centralized web-based platform for phishing website detection.
3. To allow users to submit website URLs for automated security analysis.
4. To use machine learning techniques to identify suspicious URL patterns.
5. To apply TF-IDF feature extraction for converting URLs into machine learning features.
6. To use Logistic Regression for classifying URLs as safe or phishing.
7. To perform security checks such as DNS resolution, IP address identification, HTTPS verification.
8. To analyze domain information and domain age where available.
9. To identify suspicious URL structures and possible brand impersonation.
10. To provide a detailed security analysis and display the prediction result with confidence information.
11. To maintain a history of previously scanned websites for future references.
12. To provide a user-friendly interface for performing website security analysis.
13. To provide a review and feedback facility for users.
14. To support screenshot-based website analysis for identifying potential phishing websites.
15. To retrieve website details such as domain name, IP address, and website information from the identified website.
16. To improve phishing detection accuracy by combining machine learning predictions with website security checks.
17. To generate clear and understandable analysis results that help users make informed decisions about website safety

SCOPE OF THE PROJECT

- PhishGuard is a web-based application designed to detect potentially phishing Websites.
- It allows users to enter a website URL for automated security analysis.
- The system uses TF-IDF and Logistic Regression to classify URLs as safe or phishing.
- It performs security checks such as HTTPS verification, DNS resolution, IP identification, and domain analysis.
- It identifies suspicious URL patterns and possible brand impersonation.
- The system displays prediction results with confidence information through a user-friendly interface.
- It stores scan history, including URL, result, confidence, and scan time.
- It provides features such as Threat Detector, History, Methodology, Reviews, Capabilities, and About Us.
- The system supports screenshot-based website analysis to help identify potentially phishing websites.
- It retrieves domain details, IP addresses, and other website information wherever available.
- It provides a review and feedback facility for users to share their opinions about the system.
- The application helps users understand potential website security risks and encourages safer browsing practices.

Thus, the scope of the PhishGuard project covers the complete process of URL submission, phishing prediction, security analysis, result reporting, and scan history management through a single web-based application.

CHAPTER – II

SYSTEM ANALYSIS

System analysis is an important phase in the development of the PhishGuard system. It involves studying existing phishing detection methods, identifying their limitations, and defining the requirements of the proposed system.

Phishing is a cybersecurity threat in which attackers create fraudulent websites that imitate legitimate websites to steal sensitive information such as usernames, passwords, banking details, and personal data. Existing phishing detection methods include security tools, browser warnings, URL scanners, and manual verification. However, some methods provide limited security information, while manual verification is time-consuming and requires technical knowledge.

PhishGuard is designed as a web-based application that combines machine learning-based URL prediction with additional website security checks. It uses TF-IDF feature extraction and Logistic Regression to classify URLs as safe or potentially phishing. The system also performs DNS resolution, IP address identification, HTTPS verification, and domain analysis wherever applicable.

The application displays prediction results and confidence information through a user-friendly dashboard. It also provides scan history, screenshot-based website analysis, methodology details, and user reviews to improve usability and help users understand website security.

The system analysis is divided into the following major areas:

1. **Existing System**
2. **Disadvantages of Existing System**
3. **Proposed System**
4. **Advantages of Proposed System**

The system aims to make website security analysis more accessible and help users make informed decisions while browsing the internet.

2.1 EXISTING SYSTEM

The existing system for phishing website detection uses various security tools, browser warnings, online URL scanners, and manual verification methods to identify potentially harmful websites. These methods help users identify suspicious website links and protect their personal information from cyber threats. Some existing tools analyze website URLs, domain names, website reputation, and security certificates to determine whether a website may be unsafe.

Traditional phishing detection methods mainly depend on predefined rules, blacklists, domain reputation, and suspicious URL patterns. Blacklist-based systems compare the submitted URL with a list of previously identified phishing websites. If the URL is present in the blacklist, the system may warn the user about the potential threat. Browser security warnings also help users avoid accessing websites that are already recognized as dangerous.

Some advanced systems use machine learning techniques to analyse URL characteristics and classify websites as legitimate or phishing. However, the detection accuracy of these systems depends on the quality of the training dataset, the features used, and the ability to identify newly created phishing websites. Certain tools provide only a basic prediction result without explaining the security-related factors behind the decision.

In addition, users may need to access multiple websites or security services to check domain information, HTTPS certificates, DNS details, and other security indicators. Manual verification requires time and technical knowledge, making it difficult for ordinary users to identify sophisticated phishing attacks. Some existing tools also lack features such as scan history, screenshot-based website analysis, and a centralized dashboard for displaying security information.

Therefore, the existing system has certain limitations in providing comprehensive website security analysis through a single, user-friendly platform.

2.2 PROPOSED SYSTEM

The proposed system, PhishGuard, is a web-based phishing website detection and security analysis application developed to help users identify potentially unsafe websites. The system combines machine learning techniques with additional security verification methods to analyse website URLs and provide understandable prediction results. It aims to improve user awareness of phishing threats and support safer browsing practices.

PhishGuard uses TF-IDF character n-gram feature extraction to convert website URLs into numerical features. These features are processed by a Logistic Regression machine learning model to predict whether a submitted URL is safe or potentially phishing. The system displays the prediction result along with confidence information, allowing users to understand the classification provided by the model.

In addition to machine learning prediction, the system performs several security checks, including DNS resolution, IP address identification, HTTPS certificate verification, domain analysis, and brand impersonation detection wherever applicable. These checks provide additional information about the website and help users identify suspicious characteristics. The system also retrieves website information when accessible to support the overall security analysis.

PhishGuard provides a user-friendly dashboard that presents the analysis results and security findings in an organized format. It maintains scan history so that users can review previously analysed URLs. The application also supports screenshot-based website analysis, enabling users to submit website screenshots for additional inspection. Methodology information and user reviews are included to improve the usability and understanding of the application.

The proposed system integrates multiple website analysis features into a single platform, reducing the need to depend on several separate security tools. Its simple interface makes website security analysis more accessible to users without advanced technical knowledge.

CHAPTER - III
SYSTEM SPECIFICATION

System specification describes the hardware and software requirements needed to develop and run the PhishGuard phishing website detection system. It defines the minimum resources required for the proper functioning of the application, including URL analysis, machine learning prediction, security verification, screenshot analysis, and dashboard operations.

3.1 HARDWARE SPECIFICATION

The hardware requirements specify the physical components needed to execute the PhishGuard application efficiently.

- **Processor:** Intel Core i3 or equivalent and above.
- **RAM:** Minimum 4 GB.
- **Hard Disk:** Minimum 500 MB of free storage space for project files
- **Display:** Standard monitor with a resolution of 1366 × 768 or above.
- **Input Devices:** Keyboard and mouse.
- **Network:** Internet connection for website access, DNS resolution, and online security checks.

The hardware requirements of the PhishGuard system are designed to ensure smooth execution of the application and efficient processing of website URLs. A computer with sufficient memory, processing power, and storage is required to run the Flask application, load the trained machine learning model, and store scan history.

3.2 SOFTWARE SPECIFICATION

The software requirements specify the programming languages, libraries, tools, and platforms used to develop and execute the system.

- **Operating System:** Windows 10 or Windows 11.
- **Programming Language:** Python.

- **Frontend Technologies:** HTML, CSS, and JavaScript.
- **Backend Framework:** Flask.
- **Machine Learning Library:** Scikit-learn.
- **Feature Extraction:** TF-IDF Vectorizer.
- **Machine Learning Algorithm:** Logistic Regression.
- **Data Processing:** Pandas and NumPy.
- **Model Storage:** Joblib for saving and loading trained machine learning models.
- **Database:** SQLite for storing scan history and user reviews.
- **Website Analysis:** Analyzing accessible website content.
- **Development Environment:** Visual Studio Code
- **Web Browser:** Google Chrome, Microsoft Edge, or Mozilla Firefox.

3.3 Software Description

PhishGuard is a web-based phishing website detection and security analysis application developed using Python and the Flask framework. It provides a user-friendly interface for submitting website URLs and uploading website screenshots for analysis.

The application uses TF-IDF character n-gram feature extraction and a Logistic Regression model to predict whether a submitted URL is safe or potentially phishing. It also performs additional security checks, including DNS resolution, IP address identification, HTTPS verification, domain analysis, and brand impersonation detection wherever applicable.

HTML, CSS, and JavaScript are used to design the user interface, while Flask handles application requests and processing. SQLite stores scan history and user reviews. Joblib is used to save and load the trained machine learning model and vectorizer.

SYSTEM DESIGN

4.1 DATABASE DESIGN

Database design is an important part of the PhishGuard system, as it helps organize and store the information generated during website security analysis. The system uses an SQLite database to maintain scan history and user reviews. The database contains two main tables: **scan_history** and **reviews**. The scan_history table stores the results of website scans, while the reviews table maintains feedback provided by users.

1. SCAN_HISTORY TABLE

The scan_history table stores information about the URLs analysed by the PhishGuard application.

Field Name	Description
id	Unique scan identification number
url	Website URL submitted for analysis
result	Predicted website classification
confidence	Prediction confidence value
scan_time	Date and time of the scan

Table 4.1.1: Structure of Scan History Table

2. REVIEWS TABLE

The reviews table stores feedback submitted by users about the PhishGuard application. It includes the review ID, user's name, rating, review comments, and review submission time. This table helps collect user feedback and understand their experience with the application.

Field Name	Description
id	Unique review identification number
name	Name of the reviewer
rating	Rating given by the user
review	Feedback or comments
review_time	Date and time of review submission

Table 3.2: Structure of User Reviews Table

4.2 DATAFLOW DIAGRAM

The Data Flow Diagram (DFD) represents the flow of data within the PhishGuard phishing website detection system. It explains how data is received from the user, processed by the machine learning model, analysed using security verification methods, and presented as a final result.

Data Flow of PhishGuard:

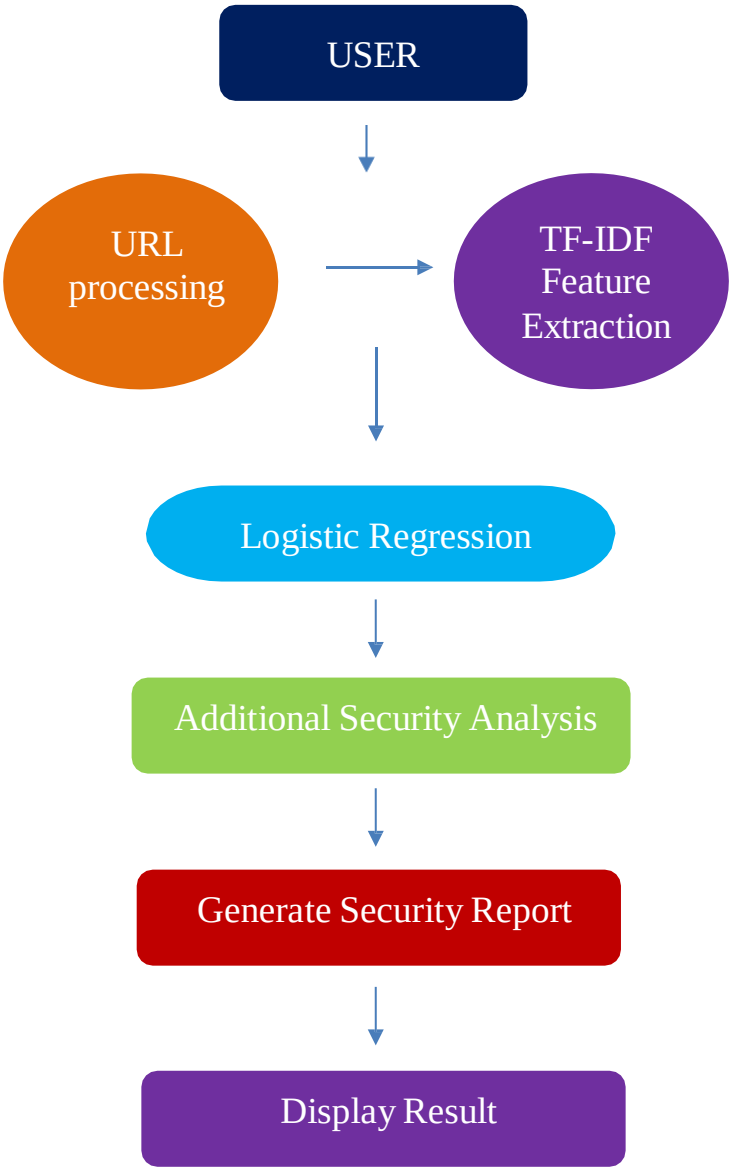


Figure 4.2.1: Data Flow Diagram of PhishGuard Phishing Website Detection System

The above Data Flow Diagram illustrates the overall workflow of the PhishGuard system. It shows how the user submits a website URL or screenshot, which is processed by the system for phishing detection.

CHAPTER – V

SYSTEM IMPLEMENTATION

5.1 MODULE DESCRIPTION

System implementation is the phase in which the design and specifications of the PhishGuard system are converted into a working application. The system is developed as a web-based phishing website detection and security analysis application using Python and the Flask framework. It integrates machine learning techniques, website security checks, database connectivity, and a user-friendly dashboard to help users identify potentially unsafe websites.

- **Threat Detection Module** – Accepts the website URL from the user and analyzes it using machine learning model and security checks.
- **Machine Learning Module** – Uses TF-IDF feature extraction and
- Logistic Regression to classify the submitted URL as safe or phishing.
- **Security Analysis Module** – Performs checks such as HTTPS verification, DNS resolution, IP address identification, domain information, and suspicious URL pattern analysis.
- **Result Module** – Displays the prediction result, confidence information, and security details to the user.
- **History Module** – Stores and displays previously scanned website URLs, results, confidence values, and scan times.
- **Review Module** – Allows users to submit ratings and feedback about the PhishGuard system.
- **Methodology and Information Module** – Provides information about the detection methodology, system capabilities, and project details.

The implementation of PhishGuard integrates machine learning prediction, website security verification, screenshot analysis, database connectivity, and dashboard visualization into a single web-based application. The system provides users with clear prediction results and relevant security findings through a user-friendly interface.

5.2 FORM DESIGN

PhishGuard Home Page

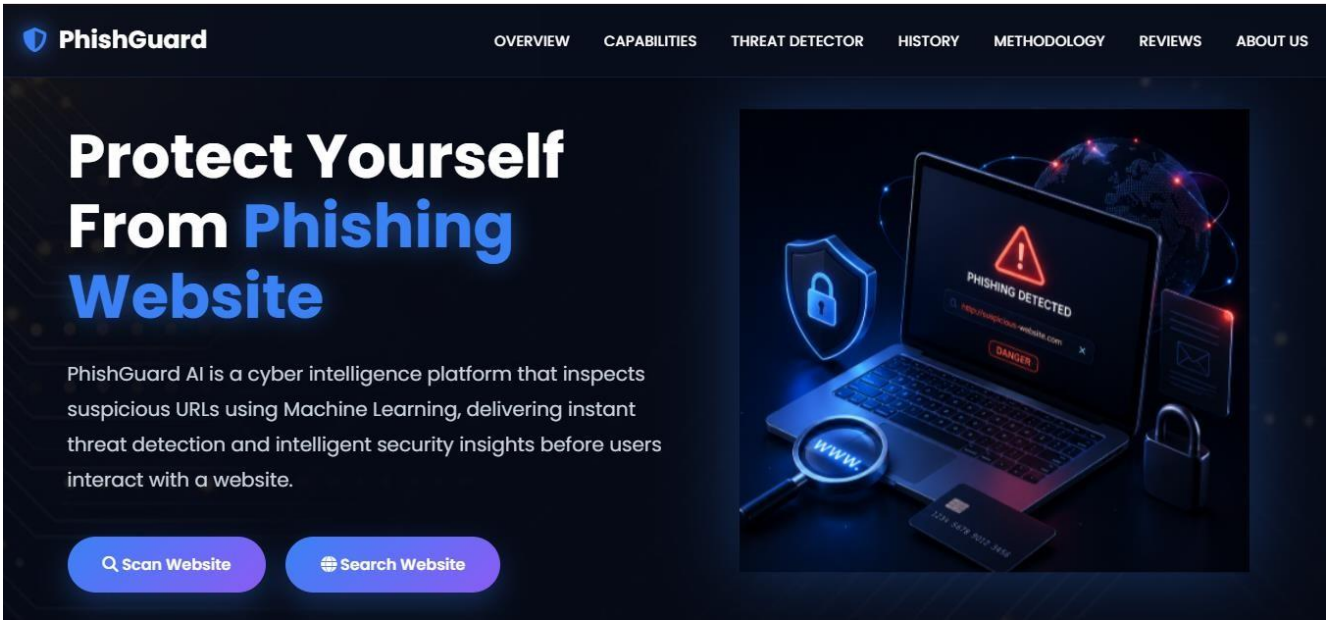


Figure 5.1 – PhishGuard Home Page

Module Overview: Displays the home page of PhishGuard and provides access to its main features.

Threat Detector – URL Submission Form

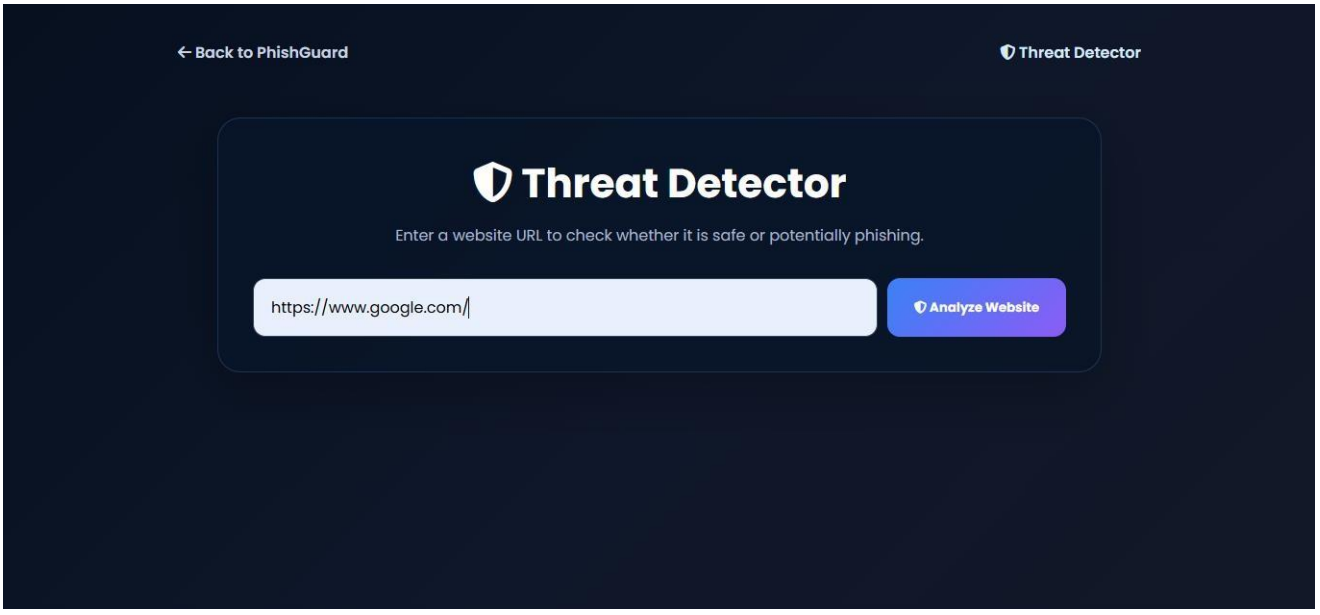


Figure 5.2 – Threat Detector – URL Submission Form

Module Overview: Allows users to enter a website URL for phishing detection and security analysis.

Website Analysis Report – Website Information:

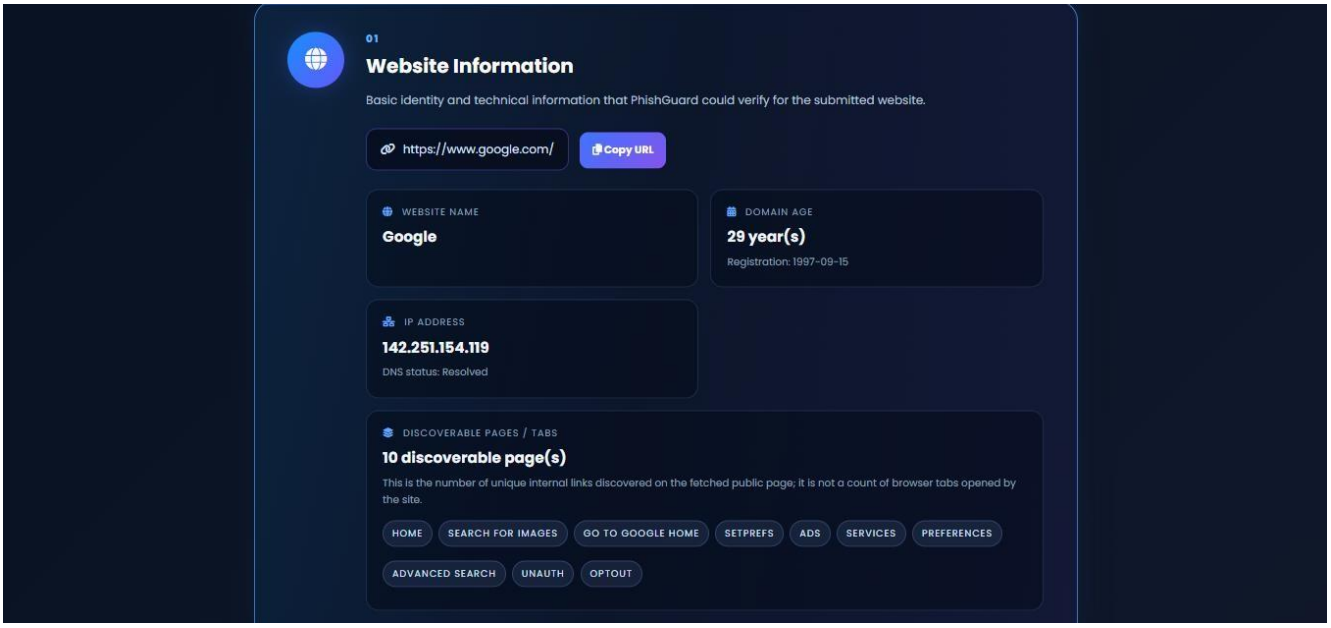


Figure 5.3 – Website Analysis Report – Website Information

Module Overview: Displays website details such as website name, domain age, IP address, and discoverable pages.

Website Analysis Report – Website Purpose and Users:

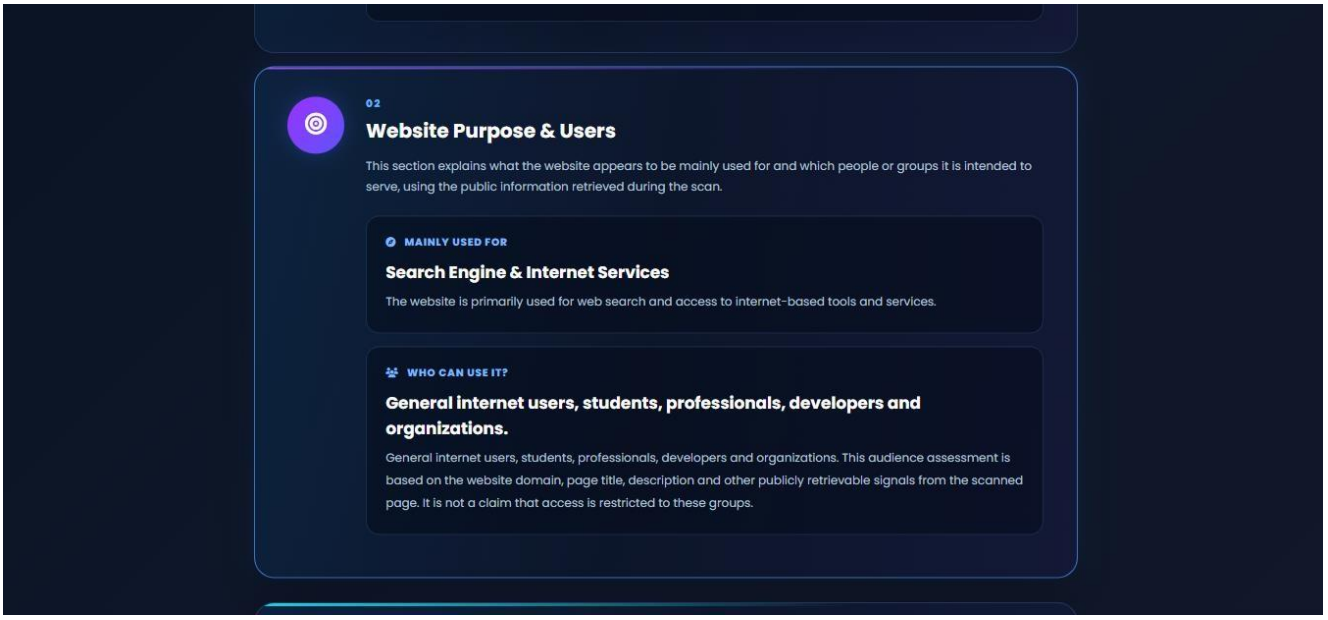


Figure 5.4 – Website Analysis Report – Website Purpose and Users

Module Overview: Describes the purpose of the website and its intended users.

Website Analysis Report – Prediction and Security Result:

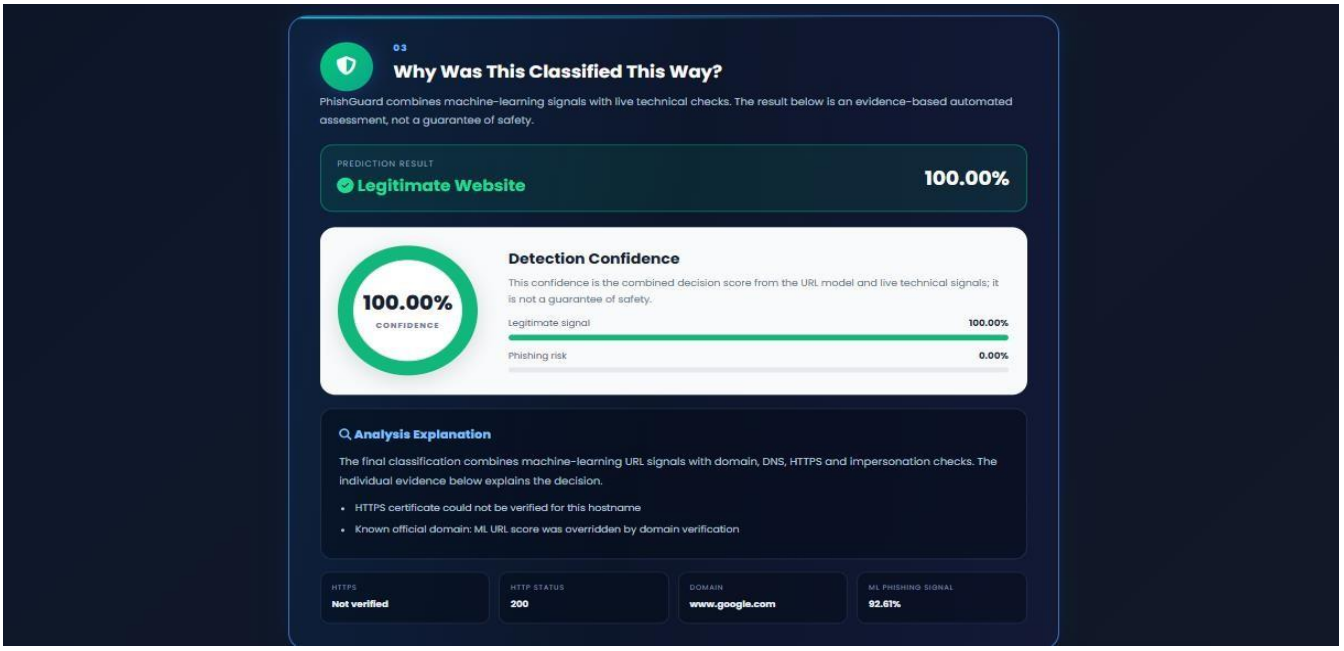


Figure 5.5 – Website Analysis Report – Prediction and Security Result

Module Overview: Displays the prediction result, confidence percentage, analysis explanation, HTTPS status, HTTP status, domain, and ML phishing signal.

Scan History Form:



Figure 5.6 – Scan History Page

Module Overview: Displays previously scanned URLs along with their results, confidence percentages, and scan times.

Threat Detector Module

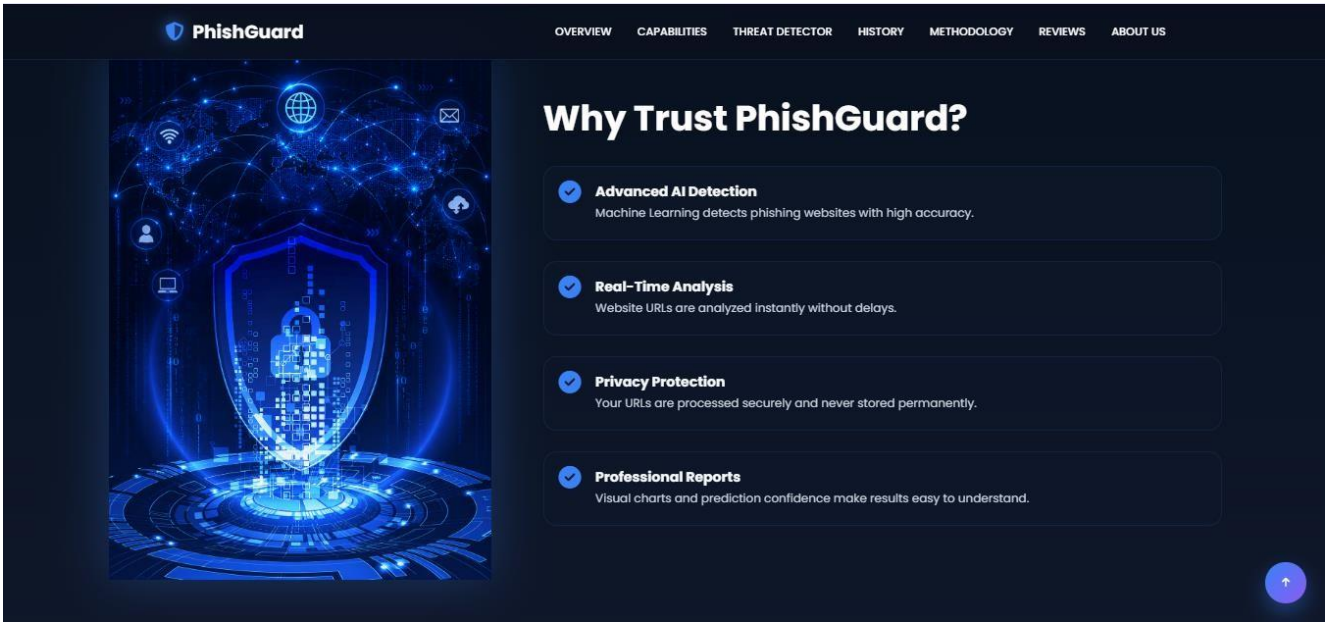


Figure 5.7: Threat Detector Module – PhishGuard Website Interface

Module Overview: Displays PhishGuard's security features for detecting phishing websites.

Search Websites Module:

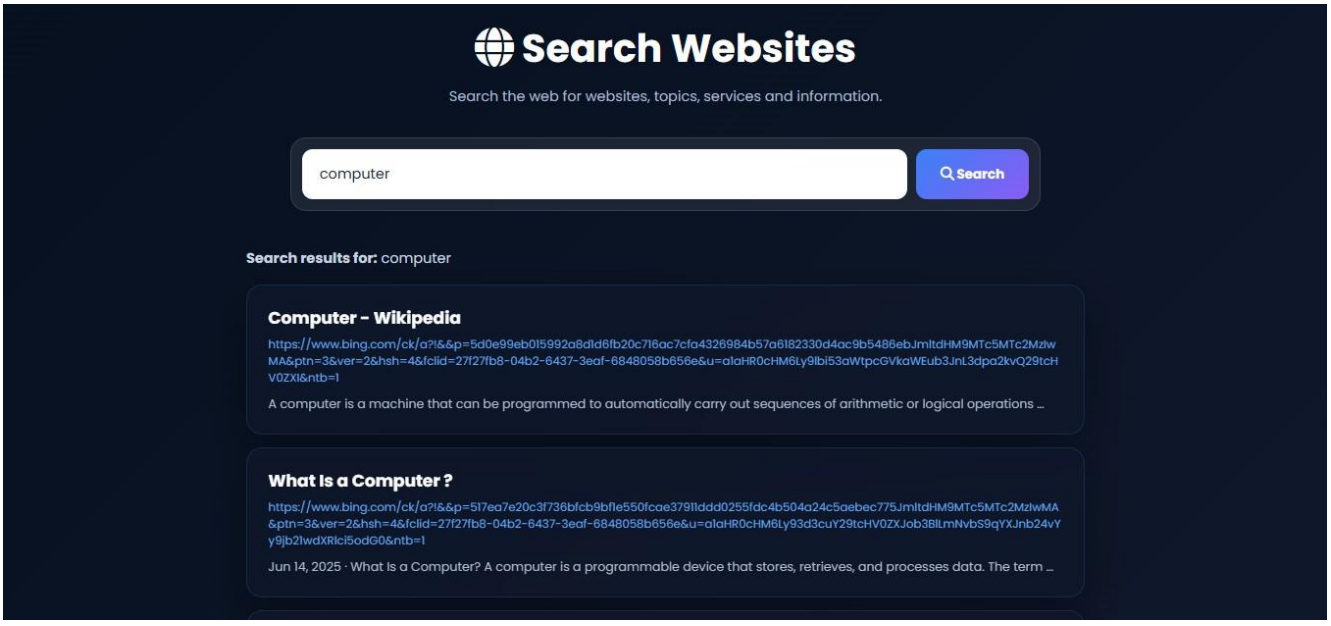


Figure 5.8: Web Search Module – Search Websites Interface

Module Overview: This interface allows users to input search queries into the system and displays the fetched search results from the web in real-time

User Review and Feedback Form

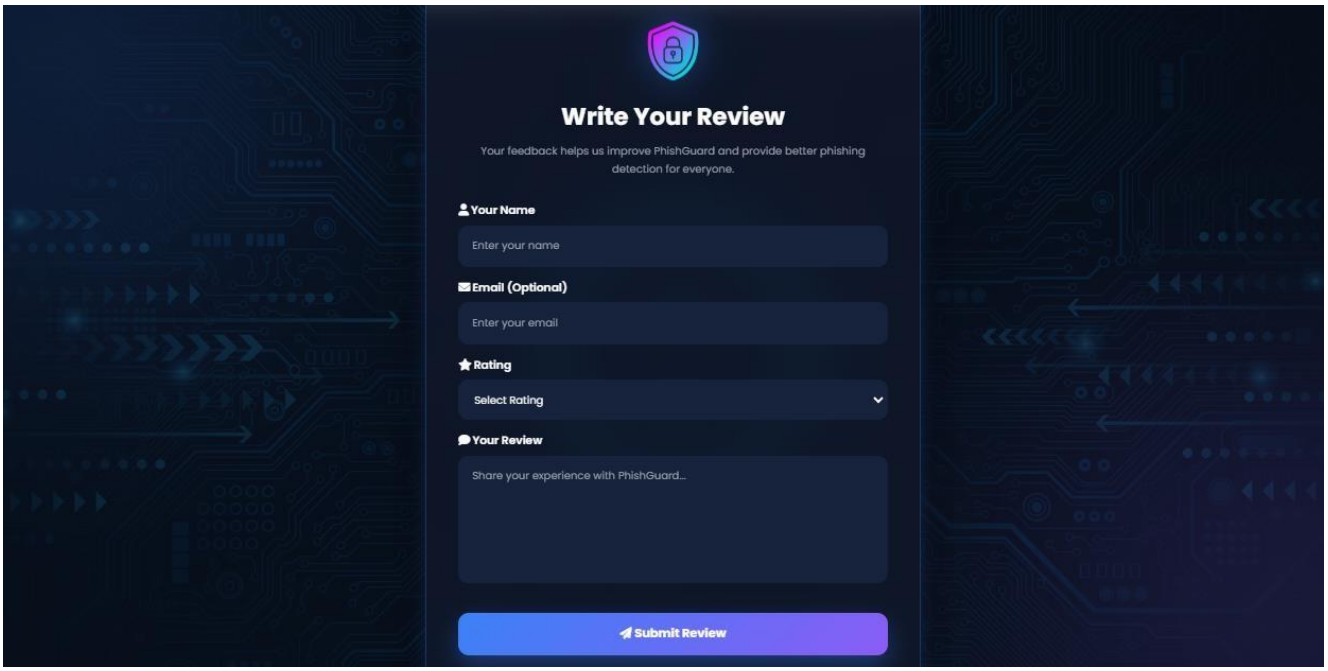


Figure 5.8: Feedback Module – PhishGuard User Review Interface

Module Overview: "This interface allows users to submit their feedback, ratings, and reviews to help improve the system's phishing detection services."

About Us Page

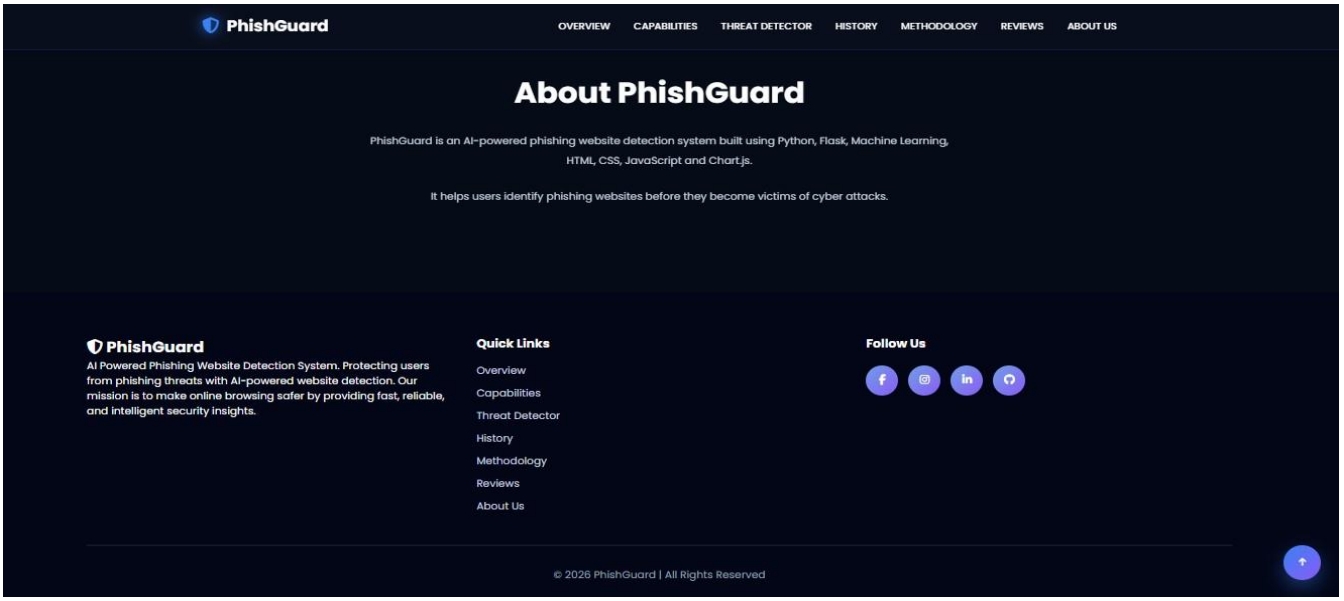


Figure 5.9: Information Module – PhishGuard About Us Interface

Module Overview: This interface provides an overview of the PhishGuard system, detailing its core features, underlying technologies, and navigation links.

CHAPTER - VI
SYSTEM TESTING

System testing is performed to verify whether the PhishGuard application works correctly according to the specified functional and technical requirements. Different functions of the system are rigorously evaluated to identify potential errors and ensure that website detection, security analysis, database management, and user interface components operate properly under standard working conditions. The main testing activities carried out in this phase include:

- **URL Validation Testing** – Checks whether the entered website URL format is valid, sanitized, and properly processed by the system before initiating the analysis workflow.
- **Prediction Testing** – Verifies whether the trained machine learning model accurately extracts domain features and correctly classifies input URLs as either legitimate or phishing threats.
- **Security Analysis Testing** – Checks whether security components such as HTTPS verification, DNS record lookup, IP address tracking, domain age analysis, and suspicious pattern detection execute correctly.
- **Database Testing** – Verifies that critical application data, including historical scan logs, user search queries, and submitted review feedback, are accurately stored and retrieved without data corruption.
- **Interface Testing** – Ensures that interactive elements such as search boxes, submit buttons, review forms, navigation links, and dynamic visual result pages function seamlessly across different display viewports.
- **Error Handling Testing** – Evaluates how the application responds to edge cases such as malformed URLs, empty search inputs, network connection timeouts, or unavailable target server information by delivering informative error messages.
- **Performance & Speed Testing** – Assesses the system response time during real-time feature extraction and prediction output to guarantee a fast user experience.

CHAPTER – VII
CONCLUSION AND FUTURE SCOPE

CONCLUSION

The PhishGuard – Intelligent Phishing Website Detection and Security Analysis System is developed as a web-based application to help users identify potentially phishing websites. Phishing websites can look similar to legitimate websites and may be used to collect sensitive user information. PhishGuard provides an automated approach to analyze such websites.

The system allows users to enter a website URL and perform security analysis. TF-IDF feature extraction is used to convert the URL into machine learning features, and Logistic Regression is used to classify the website as safe or phishing. In addition to machine learning prediction, the system performs security checks such as HTTPS verification, DNS resolution, IP address identification, domain information, and suspicious URL pattern analysis.

The application is developed using Python, Flask, HTML, CSS, JavaScript, Scikit-learn, and SQLite. The system also provides features such as Threat Detector, History, Methodology, Reviews, Capabilities, and About Us. The History section allows users to view previously analyzed websites and their results. Furthermore, the embedded review mechanism enables continuous user feedback collection to refine detection accuracy and improve overall user satisfaction across diverse browsing environments.

Overall, PhishGuard provides a simple and centralized platform for phishing detection, URL analysis, security checking, and result management. The project demonstrates the practical use of machine learning and web technologies in cybersecurity and can be further enhanced with advanced phishing detection techniques in the future.

FUTURE SCOPE

The PhishGuard system can be further enhanced with advanced features to improve phishing detection and security analysis.

The future enhancements may include:

- **Screenshot-Based Detection** – Analyze website screenshots using image processing and machine learning techniques to identify phishing websites based on their visual appearance.
- **Deep Learning** – Implement advanced deep learning models to improve the detection of complex and newly created phishing websites.
- **Real-Time Threat Intelligence** – Integrate real-time threat databases to obtain updated information about malicious domains and URLs.
- **Browser Extension** – Develop a browser extension that can automatically analyze websites while users are browsing.
- **Email Phishing Detection** – Extend the system to analyze suspicious emails and identify phishing links or messages.
- **QR Code Analysis** – Add QR code scanning functionality to detect potentially malicious URLs hidden inside QR codes.
- **Mobile Application** – Develop a mobile version of PhishGuard to provide phishing detection on smartphones.

Thus, these future enhancements can extend PhishGuard into a more comprehensive cybersecurity platform for detecting different types of phishing threats.

BIBLIOGRAPHY

- **Python Software Foundation**, Python
- Documentation, Official Reference Manuals for Python Programming Language and Core Standard Libraries.
- **Pallets Projects**, Flask Documentation, Web Application Framework for Python-based Backend and API Development.
- **Scikit-learn Developers**, Scikit-learn User Guide, Machine Learning Library for Predictive Data Analysis and Model Training in Python.
- **SQLite Development Team**, SQLite Documentation, Embedded Relational Database Management System for Local Data Storage.
- **Pandas Development Team**, Pandas Documentation, Open Source Data Structures and Data Analysis Tools for Python.
- **Beautiful Soup Documentation**, Beautiful Soup Documentation, HTML/XML Parsing Library for Extracting Data from Web Pages.
- **Requests Developers**, Requests Documentation, HTTP Library for Handling API Requests and Fetching Web Page Content in Python.
- **OWASP Foundation**, OWASP Web Security Resources, Guidelines for Identifying Web Vulnerabilities and Phishing Threats.
- **Phishing Website Detection Research Materials**, Machine Learning and Cybersecurity References, Research Papers on URL Analysis and Classification.
- **PhishGuard Database**, PhishGuard Documentation, Open Source Phishing Feed and Verification Platform for Threat Intelligence.
- **NumPy Developers**, NumPy Documentation, Fundamental Package for Scientific Computing and Array Operations in Python.
- **Python Standard Library**, URLLib Documentation, URL Handling, Parsing, and Network Protocol Modules.
- **Python-WHOIS Developers**, PyPI WHOIS Documentation, Domain Name Registration Lookup and Domain Age Verification Library.

APPENDIX

SAMPLE CODE:

PHISHGUARD – app.py SOURCE CODE

Flask Setup and Machine Learning Model

```
from flask import Flask, render_template, request, redirect import joblib, re, sqlite3,
socket, requests, ssl.
from urllib. parse import URL parse from difflib import SequenceMatcher app = Flask(
_____name_)
model = joblib.load("url_model.pkl") vectorizer = joblib.load("vectorizer.pkl")
OFFICIAL_DOMAINS = {
"google.com", "amazon.com", "amazon.in", "microsoft.com", "apple.com",
"github.com", "paypal.com", "openai.com", "chatgpt.com"
}
SUSPICIOUS_WORDS = {
"login", "verify", "account", "password", "payment", "secure", "confirm", "update"
}
```

URL Validation and Security Checks

```
def normalize_url(url): url = url.strip()
if not re.match(r"^https?://", url, re.I):
url = "https://" + url return url

def parse_domain(url):
return (urlparse(normalize_url(url)).hostname or "").lower() def is_valid_url(url):
try:
p = urlparse(normalize_url(url))
return p.scheme in {"http", "https"} and "." in (p.hostname or "") except Exception:
return False
def registered_domain(host): parts = host.split(".")
if len(parts) >= 3 and ".".join(parts[-2:]) in {
"co.in", "co.uk", "com.au"
```

```
}:
return ".".join(parts[-3:])
return ".".join(parts[-2:]) if len(parts) >= 2 else host def structural_signals(url):
risk, reasons = 0.0, []
p = urlparse(normalize_url(url)) if len(url) > 100:
risk += 0.12 reasons.append("Long URL")
if "@" in url: risk += 0.20
reasons.append("Contains @ symbol") if url.count("-") >= 4:
risk += 0.08

reasons.append("Excessive hyphens") if p.scheme == "http":
risk += 0.08 reasons.append("Uses HTTP")
if any(w in url.lower() for w in SUSPICIOUS_WORDS): reasons.append("Suspicious
keywords")
return min(risk, 0.90), reasons def brand_impersonation(domain):
reg = registered_domain(domain) if reg in OFFICIAL_DOMAINS:
return False
brands = ["amazon", "google", "microsoft", "apple", "paypal"] name = reg.split(".")[0]
return any(
name != b and SequenceMatcher(None, name, b).ratio() >= 0.80 for b in brands
)
def resolve_ip(domain): try:
return socket.gethostbyname(domain) except OSError:
return None
def certificate_matches(url):
p = urlparse(normalize_url(url)) if p.scheme != "https":
return None

try:
context = ssl.create_default_context()
```

```
with socket.create_connection((p.hostname, 443), timeout=3) as sock: with
context.wrap_socket(sock, server_hostname=p.hostname) as
secure:
return secure.getpeercert() is not None except Exception:
return False
```

Machine Learning-Based Phishing Detection

```
def analyze_url(url):
clean = normalize_url(url) domain = parse_domain(clean)
official = registered_domain(domain) in OFFICIAL_DOMAINS ip = resolve_ip(domain)
features = vectorizer.transform([clean.lower()]) probs = model.predict_proba(features)[0]
classes = list(model.classes_)
bad = float(probs[classes.index("bad")]) if "bad" in classes else 0.5
good = float(probs[classes.index("good")]) if "good" in classes else 1-bad structural,
reasons = structural_signals(clean)
impersonation = brand_impersonation(domain) certificate = certificate_matches(clean)
if official and ip:
risk = min(structural * 0.35, 0.25) result = "Safe"
elif not ip or impersonation:
risk = 1.0 if not ip else 0.85 result = "Phishing"
else:
risk = bad * 0.55 + structural * 0.25 if certificate is False:
risk += 0.08
risk = min(max(risk, 0), 1)
result = "Phishing" if risk >= 0.50 else "Safe"  confidence = (risk if result == "Phishing"
else 1-risk) * 100 return {
"classification": result, "confidence": round(confidence, 2),
"safe_value": round((1-risk)*100, 2),
"phishing_value": round(risk*100, 2), "domain": domain,
"ip_address": ip or "Not available",
```

"dns_status": "Resolved" if ip else "Not resolved", "https_status": "Verified" if certificate else "Not verified", "ml_phishing": round(bad*100, 2),
"ml_safe": round(good*100, 2), "reasons": reasons
}
Database Connectivity
def init_db():
with sqlite3.connect("history.db") as conn: conn.execute("""
CREATE TABLE IF NOT EXISTS scan_history(
id INTEGER PRIMARY KEY AUTOINCREMENT,
url TEXT, result TEXT,
confidence REAL, scan_time TEXT
) """)
conn.execute("""
CREATE TABLE IF NOT EXISTS reviews(
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT, rating INTEGER, review TEXT, review_time TEXT
) """)
def save_history(url, result, confidence): with sqlite3.connect("history.db") as conn:
conn.execute("""
INSERT INTO scan_history
(url, result, confidence, scan_time)
VALUES (?, ?, ?, datetime('now','localtime')) """, (url, result, confidence))
def get_history():
with sqlite3.connect("history.db") as conn: return conn.execute("""
SELECT url, result, confidence, scan_time
FROM scan_history ORDER BY id DESC LIMIT 6 """).fetchall()
init_db()
Flask Routes and Execution
@app.route("/") def home():

```
return render_template("index.html") @app.route("/detector", methods=["GET",
"POST"]) def detector_page():
url, analysis, error = "", None, None if request.method == "POST":
url = request.form.get("url", "").strip() if not url or not is_valid_url(url):
error = "Please enter a valid website URL." else:
try:
analysis = analyze_url(url) save_history(
url, analysis["classification"], analysis["confidence"]
)
except Exception:
error = "Unable to analyze this website." return render_template(
"detector.html", entered_url=url, analysis=analysis, error=error
)
@app.route("/history")
def history_page():
return render_template("history.html", history=get_history())
@app.route("/submit_review", methods=["POST"]) defsubmit_review():
name=request.form.get("name", "Anonymous") review=request.form.get("review", "")
rating=request.form.get("rating", 5)

withsqlite3.connect("history.db") as conn: conn.execute("""
INSERT INTO reviews(name, rating, review, review_time) VALUES (?, ?, ?,
datetime('now','localtime'))
""",(name, rating, review)) returnredirect("/review?submitted=1")
@app.route("/review") defreview_page():
return render_template( "review.html",
submitted=request.args.get("submitted") == "1"
)
if____name_== "__main_": app.run(debug=True)
```